Machine Learning I

Bachelor in Data Science and Engineering

Assignment 2: Reinforcement Learning

Students:

Eloi Rodríguez Rodríguez
ID: 100496948
100496948@alumnos.uc3m.es

Leyre Ruiz de Alegría Bravo
ID: 100497014
100497014@alumnos.uc3m.es

Universidad Carlos III de Madrid
April 2026

Contents

1 Phase 1: Manual State Discretization and Q-Learning

1.1 Step 1: Manual State Discretization

We discretized all 8 environment variables using fixed bins combined via mixed-radix encoding into a single integer state identifier:

$$\text{state\_id} = i_0 + i_1 N_0 + i_2 N_0 N_1 + \cdots \qquad N_{\text{states}} = 5 \times 5 \times 3 \times 3 \times 3 \times 3 \times 2 \times 2 = 8100.$$

Variable	# Bins	Edges	Justification
`x_position`	5	$\{-0.5, -0.1, 0.1, 0.5\}$	Alignment with landing pad ($x = 0$).
`y_position`	5	$\{0.1, 0.4, 0.8, 1.2\}$	More resolution near the ground.
`x_velocity`	3	$\{-0.2, 0.2\}$	Left / still / right.
`y_velocity`	3	$\{-0.4, -0.1\}$	Falling fast / normal / hovering.
`angle`	3	$\{-0.2, 0.2\}$	Tilted left / upright / tilted right.
`angular_velocity`	3	$\{-0.3, 0.3\}$	Rotating / stable / rotating.
`left_leg`	2	—	Boolean contact flag.
`right_leg`	2	—	Boolean contact flag.

Table 1: Manual discretization scheme.

Q1. Which state variables did you choose to discretize, and why are they relevant for the landing task? All 8 variables were discretized. `x_position` and `y_position` determine whether the lander is above the pad and how close it is to the ground. These are the two most critical spatial quantities. `x_velocity` and `y_velocity` are needed to anticipate collisions and control the descent speed. `angle` and `angular_velocity` capture the lander's orientation and rotational stability, both essential for a controlled touchdown. The two leg-contact flags are the terminal success signal and allow the agent to learn to maintain thrust until both legs touch the ground.

Q2. How did you define the discretization ranges or thresholds for each variable? Thresholds were chosen from domain knowledge and inspection of typical ranges. For `x_position`, edges at ± 0.1 delimit the narrow landing zone and ± 0.5 mark far off-center regions. For `y_position`, denser edges near the ground (0.1, 0.4) give more resolution where precise control matters. Velocity and angular variables use symmetric edges around zero to separate negative, near-zero and positive regimes. The leg flags are already binary, so a single threshold at 0.5 suffices.

Q3. How does the size of the state space affect learning performance and convergence? A smaller state space reduces resolution: states that require different actions are merged, limiting the quality of the learned policy. A larger state space leaves many states unvisited, keeping their Q-values at zero and slowing convergence. With $N = 8100$ states and 5000 training episodes our agent visited $\approx 35\%$ of the space, sufficient for the most frequent states to converge while keeping the table manageable.

1.2 Step 2: Reward Function Design

The shaped reward adds the following terms to the original Gymnasium reward:

$$r = r_{\text{raw}} - 0.10\,|x| - 0.10\,|v_y| - 0.20\,|\theta| - 0.05\,|\dot{\theta}| + 0.10\,(\text{left\_leg} + \text{right\_leg}).$$

Q1. What modifications did you introduce to the original reward function? Four penalty terms proportional to horizontal displacement, vertical speed, tilt angle, and angular velocity were added, together with a small bonus for leg contact. Coefficients were kept small so the shaping remains secondary to the environment reward.

Q2. Which aspects of the lander's behavior are encouraged or penalized? The shaped reward **penalizes** off-center positions, fast descent, large tilt, and fast rotation. It **rewards** ground contact with either leg, encouraging a stable and centered touchdown.

Q3. How did your reward shaping affect the stability and convergence of the learning process? The shaped reward did not improve final performance. Per-step penalties accumulate over longer episodes, biasing the agent toward quick termination rather than successful landing. Evaluation in PLAY mode under the raw reward confirmed the shaped agent performed no better than the baseline, so the raw-reward Q-table was selected as the best Phase 1 model.

1.3 Step 3: Q-Learning Implementation

Bellman update.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]. \tag{1}$$

Q1. Explain how the Bellman update equation is applied in your implementation. After each macro-action (`ACTION_REPEAT` steps), the agent observes the accumulated reward r and next state s', then applies Equation 1. The TD error $\delta = r + \gamma \max_{a'} Q(s',a') - Q(s,a)$ corrects the current estimate: positive δ means the outcome was better than expected and $Q(s,a)$ increases; negative δ decreases it. Repeated over many episodes this drives Q toward the optimal Q^*.

Q2. What is the role of the discount factor γ, and how does it influence the learned policy? γ weights future rewards relative to immediate ones: a reward k steps ahead contributes γ^k of its value. With $\gamma = 0.95$ the large terminal landing reward propagates back through the Q-table over many updates, enabling long-horizon planning. A smaller γ would make the agent myopic and unable to plan the full descent.

Q3. Why is an ε-greedy strategy necessary, and what happens if ε is too high or too low? Without exploration the agent never visits states unreachable by its initial policy. The exploration rate decays exponentially:

$$\varepsilon_e = \varepsilon_{\min} + (\varepsilon_{\max} - \varepsilon_{\min})\, e^{-\lambda e}.$$

Too high: the agent keeps exploring and never exploits the learned policy. Too low: the agent commits early to a suboptimal policy and gets stuck. The exponential schedule provides broad exploration early and shifts to exploitation as the Q-table matures.

1.4 Step 4: Training and Hyperparameter Tuning

Three experiments of 5000 episodes were run. All used $\gamma = 0.95$, $\varepsilon_{\max} = 1.0$, $\varepsilon_{\min} = 0.05$, `ACTION_REPEAT`$= 5$.

Experiment	Reward	α	λ	Avg. reward (last 500 ep.)
Exp. 1 – Raw reward	raw	0.2	0.002	≈ 0 to $+25$
Exp. 2 – Shaped	shaped	0.2	0.002	≈ -100 to -50
Exp. 3 – Shaped, tuned	shaped	0.1	0.001	≈ -100 to -50

Table 2: Phase 1 experiments. Exp. 2–3 rewards include shaping penalties and are not directly comparable to Exp. 1.

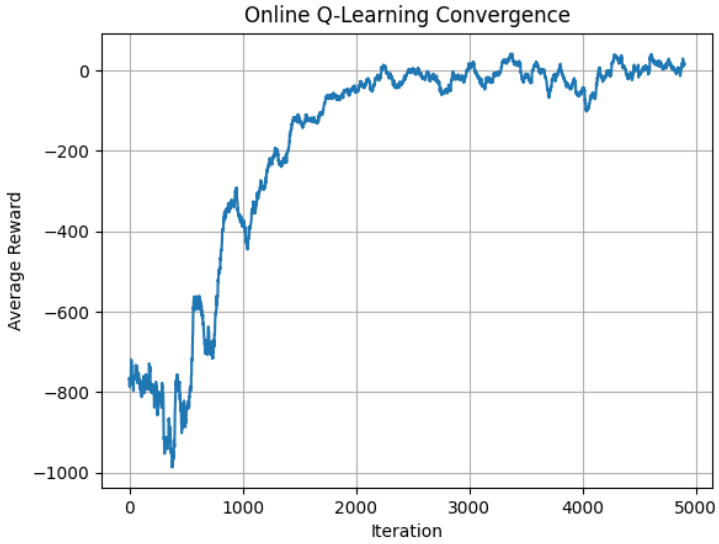

Figure 1: Exp. 1: raw reward, $\alpha = 0.2$, $\lambda = 0.002$.

Q1. Which hyperparameters did you tune, and what values provided the best results? α and λ were the main parameters varied. High $\alpha > 0.3$ caused Q-value instability. The combination $\alpha = 0.1$, $\lambda = 0.001$ gave the smoothest curve, but the best final performance (raw reward in PLAY mode) came from Exp. 1: $\alpha = 0.2$, $\lambda = 0.002$, raw reward.

Q2. How did the training curve evolve over time? Did the agent converge? All curves rise steeply in the first $\sim$2000 episodes as ε decays, then plateau with noise. Exp. 1 stabilizes near zero; Exp. 3 is smoother but at a lower level. Full convergence to consistent positive rewards was not reached within 5000 episodes.

Q3. What challenges did you encounter during training, and how did you address them? The main challenges were: (i) the shaped reward hurt performance — diagnosed by evaluating agents under a common raw-reward metric; (ii) high episode-to-episode variance from random initial conditions — addressed by reporting 100-episode

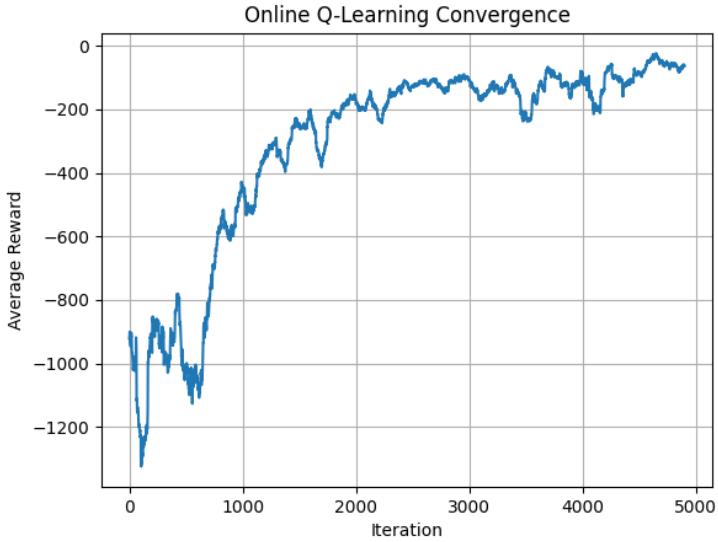

Figure 2: Exp. 2: shaped reward, $\alpha = 0.2$, $\lambda = 0.002$.

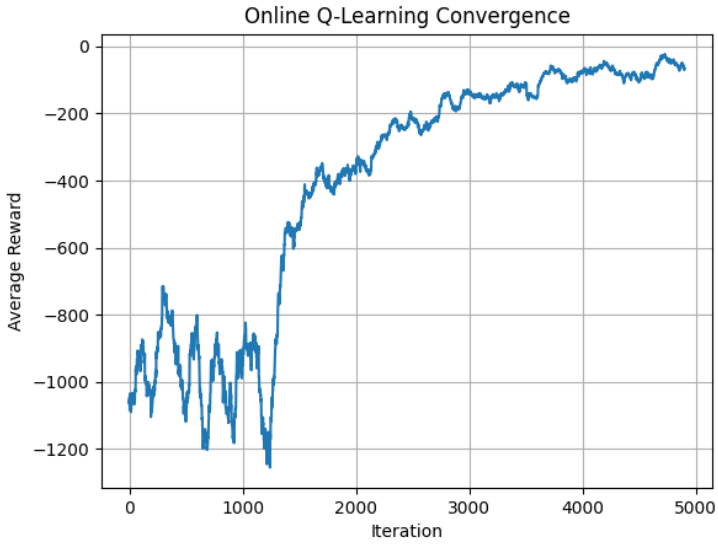

Figure 3: Exp. 3: shaped reward, $\alpha = 0.1$, $\lambda = 0.001$.

moving averages; (iii) a large fraction of the 8 100 manual states were never visited, leaving their Q-values at zero and causing occasional erratic behaviour when the agent reached one of them.

2 Phase 2: State Discretization using VQQL

2.1 Step 1: Training the Vector Quantizer

The vector quantizer was trained using `VQQL_Discretization.py` with `MiniBatchKMeans` on a mixed dataset of 50 000 state observations (25 000 from `agent_data.csv` and 25 000 from `keyboard_data.csv`, since we did not find any provided dataset). Prior to clustering,

all features were standardized with `StandardScaler` so that no variable dominates the distance computation. Three values of K were evaluated: 64, 128, and 256 clusters.

Q1. Which variables did you include in the state representation, and why? The same 8 variables used in Phase 1 were included: `x_position`, `y_position`, `x_velocity`, `y_velocity`, `angle`, `angular_velocity`, `left_leg`, and `right_leg`. these variables fully describe the lander's pose, motion, and contact state, which are the quantities the control policy must act upon. Including all of them ensures that the learned centroids capture every relevant aspect of the flight trajectory.

Q2. How does the number of clusters affect the granularity of the state space? Each cluster centroid represents a region of the continuous state space. With few clusters the regions are large and coarse: states that may require different actions are collapsed into the same discrete identifier, reducing the expressiveness of the Q-table. With many clusters the regions are smaller and finer, allowing the agent to distinguish between subtly different situations. In our experiments:

Clusters K	Q-table size	Avg. reward (last 500 ep.)
64	64×4	≈ -100 to -50
128	128×4	≈ -50 to 0
256	256×4	≈ -25 to $+25$

Table 3: Effect of the number of clusters on state granularity and final performance. All models trained for 5 000 episodes with $\alpha = 0.1$, $\gamma = 0.95$, $\lambda = 0.001$.

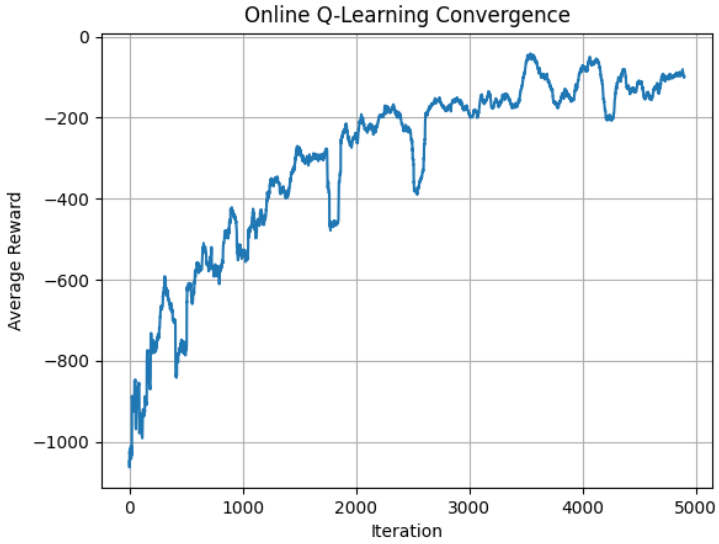

Figure 4: Training curve for $K = 64$ clusters.

Q3. What trade-offs did you observe when using a small vs. large number of clusters? Using $K = 64$ produces a compact Q-table that converges quickly in the early episodes, but the coarse state representation limits the quality of the learned policy: the

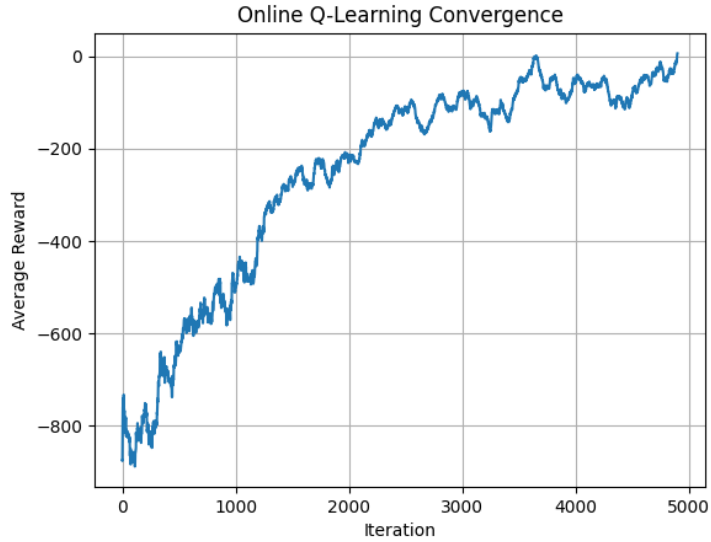

Figure 5: Training curve for $K = 128$ clusters.

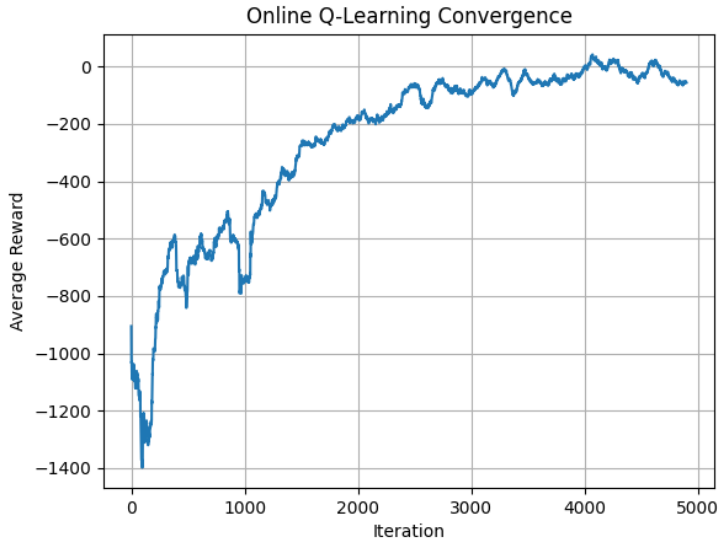

Figure 6: Training curve for $K = 256$ clusters.

agent cannot distinguish critical situations near the landing pad and the reward plateaus at a lower level. With $K = 256$ the state space is richer and the agent achieves higher final rewards, but learning is noisier in the initial phase (visible as the sharp dip around episode 200 in Figure 6) because the larger Q-table takes more episodes to populate. $K = 128$ offers a good compromise: the training curve is smooth and monotonically increasing throughout the 5 000 episodes (Figure 5), reaching performance comparable to $K = 256$ without the early instability. A smaller K converges faster but at the expense of policy quality.

2.2 Step 2: Feature Consistency and Integration

Once the quantizer was trained, it was integrated into the RL pipeline through the functions `extract_features()` and `discretize_state()` in `LunarLander_RL.py`. Three consistency requirements were verified before running any training:

1. **Same variables and same order.** The list `STATE_COLUMNS` in `VQQL_Discretization.py` and the `feature_map` dictionary in `extract_features()` both define the same 8 variables in identical order: `x_position`, `y_position`, `x_velocity`, `y_velocity`, `angle`, `angular_velocity`, `left_leg`, `right_leg`. The feature vector is built dynamically by iterating over `state_columns` (loaded from the `.pkl` file), so the order is always dictated by the file used at training time.

2. **Same scaler.** The `StandardScaler` fitted during quantizer training is saved inside the `.pkl` file and reloaded at execution time. In `discretize_state()`, the line `scaler.transform([features])` applies the stored mean and standard deviation before calling `quantizer.predict()`, guaranteeing that the scaling is identical to what was used when the centroids were computed.

3. **Valid state identifiers.** No out-of-range index errors were raised during any of the training runs (64, 128, and 256 clusters), confirming that `quantizer.predict()` always returns an integer in $[0, K-1]$ and that the Q-table is correctly sized as $K \times 4$.

Q1. Why is it important to keep the same feature representation between training and execution? The quantizer maps a continuous observation to the index of the nearest centroid in the scaled feature space. If the feature vector at execution time differs from the one used during training, the distances to the centroids are computed in a different space, so the returned cluster index has no relationship to the one the Q-table was trained with.

Q2. What issues may arise if the feature order or scaling is incorrect? First, if the feature order is wrong, each dimension of the observation is compared against the wrong centroid coordinate: for example, a velocity value would be compared against a position centroid component, producing incorrect cluster assignments. Second, if the scaler is not applied or uses different statistics (e.g. fitted on a different dataset), the distances are dominated by the variables with the largest absolute range, which breaks the balanced clustering achieved during training. In both cases the Q-table indices become meaningless and the agent behaves erratically regardless of how well it was trained.

Q3. How does the quantizer map continuous states into discrete ones? Given a continuous observation $\mathbf{x} \in R^8$, the discretization proceeds in three steps:

1. **Feature extraction.** The 8 state variables are assembled into a vector following the order stored in the `.pkl` file.

2. **Scaling.** The vector is standardized using the saved `StandardScaler`: $\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu})/\boldsymbol{\sigma}$, where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are the mean and standard deviation computed on the training dataset.

3. **Nearest-centroid assignment.** The index of the centroid $\mathbf{c}_k$ closest to $\tilde{\mathbf{x}}$ in Euclidean distance is returned as the discrete state identifier:

$$s = \arg\min_{k \in \{0,\ldots,K-1\}} \|\tilde{\mathbf{x}} - \mathbf{c}_k\|_2.$$

This index $s \in \{0, \ldots, K-1\}$ is then used as the row index into the Q-table, replacing the mixed-radix integer used in Phase 1.

2.3 Step 3: Training with VQQL Representation

Three Q-learning agents were trained using the VQQL state representation, one for each quantizer ($K \in \{64, 128, 256\}$). The Q-learning algorithm and all hyperparameters were kept identical to Phase 1 ($\alpha = 0.1$, $\gamma = 0.95$, $\lambda = 0.001$, $\varepsilon_{\max} = 1.0$, $\varepsilon_{\min} = 0.05$, `ACTION_REPEAT`$= 5$, 5 000 episodes).

Q1. How does learning behavior change compared to manual discretization? Compared to manual discretization, VQQL agents show a deeper initial dip (down to $\approx -1\,400$ for $K = 256$ vs. $-1\,000$ for manual), since larger Q-tables require more exploration to populate. However, the ascent is smoother and more monotonic: the manual agent shows irregular steps due to its $\approx 65\%$ of unvisited states, while VQQL curves rise steadily. All models stabilize near 0 in the final episodes, but VQQL exhibits lower episode-to-episode variance, reflecting more consistent coverage of the relevant state regions.

Q2. Which number of clusters provided the best performance, and why? $K = 256$ achieved the highest training-time average reward in the final episodes (≈ 0 to $+25$), followed by $K = 128$ (≈ -50 to 0) and $K = 64$ (≈ -100 to -50). The superiority of $K = 256$ during training is because with more centroids, the agent can distinguish between states that require different actions near the landing pad, where small differences in angle or velocity are critical. $K = 128$ offers a good trade-off between granularity and convergence speed, producing the smoothest and most monotonic training curve of the three.

Q3. Did you observe instability or degradation when using too many clusters? $K = 256$ shows a sharp initial dip to $\approx -1\,400$ (Figure 6) during the first 200 episodes, caused by the larger number of unvisited states whose Q-values start at zero. Despite this, the curve recovers by episode 1 000 and converges alongside the other models, confirming that 5 000 episodes is sufficient to adequately populate a 256-state Q-table.

2.4 Step 4: Evaluation and Analysis

Each trained model was evaluated in `PLAY` mode over 10 episodes with $\varepsilon = 0$ (pure exploitation). The results are summarised in Table 4.

Q1. How does the VQQL-based model compare to the manual discretization model? The best VQQL model ($K = 128$, mean $+82.5$) clearly outperforms the manual model (mean -84.9) in PLAY mode. This is because the 128 centroids are all placed in regions that the lander actually visits, so every row of the Q-table gets updated during training. The manual model, despite having 8 100 states, leaves a large fraction of them

Model	Mean reward	Std	Min	Max
VQQL $K = 64$	+14.7	172.5	−229.8	+227.7
VQQL $K = 128$	+82.5	151.5	−192.7	+254.5
VQQL $K = 256$	−83.6	138.7	−210.6	+237.6

Table 4: PLAY mode evaluation over 10 episodes for each VQQL model.

unvisited during training, so the agent has no useful information when it encounters one of those states at evaluation time.

Interestingly, $K = 256$ performed worse than $K = 128$ in PLAY mode (−83.6 vs. +82.5), even though it showed slightly better training-time rewards. The reason is that with 256 states and only 5 000 training episodes, each state is visited fewer times on average, so many Q-values do not converge to reliable estimates before evaluation. The training curve in Figure 6 reflects this: although the moving average looks reasonable, it hides a higher proportion of poorly-estimated Q-values that surface as inconsistent decisions in PLAY mode. $K = 128$ sits at the sweet spot: its Q-table is small enough to be well-populated in 5 000 episodes, yet fine enough to distinguish critical near-landing situations.

Q2. What are the advantages and disadvantages of using clustering for state representation? The main **advantages** are: (i) the state space adapts to the actual data distribution, concentrating resolution where it matters; (ii) the number of states is much smaller (64–256 vs. 8 100), reducing memory and convergence time; (iii) no domain knowledge is required to define bin edges.

The main **disadvantages** are: (i) a representative dataset must be collected before training the quantizer, adding a data-collection step; (ii) the centroids are fixed after training and cannot adapt if the agent explores new regions of the state space during Q-learning; (iii) two states that are close in Euclidean distance but require different actions (e.g. slightly different angles near touchdown) may be collapsed into the same cluster, introducing aliasing.

Q3. In which situations would you prefer a manual discretization over a learned one? Manual discretization is a better choice in some situations. If no recorded data is available, VQQL cannot be used since it requires a dataset to train the quantizer before learning can start. It is also better when the state variables have an obvious physical meaning and natural boundaries. For example, leg contact is already binary, so clustering adds no value. Another reason to prefer it is readability: a bin like "$x \in [-0.1, 0.1]$" clearly means the lander is centered over the pad, while a cluster centroid is just a point in space with no direct interpretation. Finally, if the environment is simple and the state space is small enough to be fully covered during training, a fixed grid works well. In this assignment, manual discretization is a fair baseline because the variables are well understood and their ranges are bounded. Its main weakness is that with 8 100 states and only 5 000 training episodes, most states are never visited.

3 Phase 3: Evaluation and Comparison of Models

3.1 Step 1: Comparison of Q-Learning Models

All four Q-learning agents were evaluated in `PLAY` mode over 10 episodes with $\varepsilon = 0$ (pure exploitation). The results are summarised in Table 5.

Model	States	Mean reward	Std	Min / Max
Manual discretization	8 100	−84.9	241.9	−518.6 / +243.8
VQQL $K = 64$	64	+14.7	172.5	−229.8 / +227.7
VQQL $K = 128$	128	+82.5	151.5	−192.7 / +254.5
VQQL $K = 256$	256	−83.6	138.7	−210.6 / +237.6

Table 5: PLAY mode evaluation (10 episodes, $\varepsilon = 0$) for all Q-learning models.

Q1. Which discretization method achieved better performance, and why? VQQL with $K = 128$ achieved the best mean reward (+82.5), followed by VQQL $K = 64$ (+14.7). Both the manual model and VQQL $K = 256$ obtained similar mean rewards, leaving many Q-values at zero and causing erratic behavior in unseen situations. VQQL $K = 256$, despite having a finer state representation, divides the training budget across too many states: with only 5 000 episodes, each state is visited fewer times on average, so a substantial fraction of the Q-values do not converge to reliable estimates and produce inconsistent decisions at evaluation time.

VQQL $K = 128$ succeeds because it strikes the right balance: the 128 centroids cover the relevant regions of the state space without being so numerous that the Q-table remains underpopulated after 5 000 episodes.

Q2. Which model showed more stable learning behavior? Stability can be measured by the standard deviation of PLAY-mode scores. VQQL $K = 256$ has the lowest standard deviation (138.7), followed by VQQL $K = 128$ (151.5) and VQQL $K = 64$ (172.5). The manual model is the least stable by a large margin (std = 241.9), with a worst-case score of −518.6, more than twice as bad as any VQQL model. This confirms that the data-driven state representation produces more consistent behavior across different initial conditions.

Q3. How does the size of the state space affect learning efficiency and generalization? A larger state space requires more episodes to populate the Q-table sufficiently. The manual model has 8 100 states but only 5 000 training episodes. When the agent encounters one unvisited state at evaluation time, it falls back to the first action (index 0, do nothing), which is rarely optimal. VQQL models have at most 256 states, all of which are reachable by construction since the centroids were placed in regions visited by real trajectories. This makes VQQL more sample-efficient: fewer episodes are needed to achieve good coverage, and generalization is better because nearby continuous states always map to the same centroid regardless of whether that exact point was visited during training.

Q4. Did increasing the number of states always improve performance? Why or why not? No. Going from $K = 64$ to $K = 128$ improved mean reward from

+14.7 to +82.5, but going from $K = 128$ to $K = 256$ degraded it to −83.6. This behavior illustrates the classic bias-variance trade-off in state space design: too few states introduce **bias** (state aliasing, different situations mapped to the same index), while too many states introduce **variance** (insufficient visits per state, noisy Q-values). $K = 128$ is the best compromise for this environment and training budget: enough granularity to distinguish critical situations near the landing pad, but small enough that 5 000 episodes suffice to produce reliable Q-value estimates for every state.

3.2 Step 2: Comparison with Supervised Learning Approaches

The best Q-learning model (VQQL $K = 128$, mean reward +82.5) is compared here against the two approaches developed in earlier course activities: the rule-based agent from Tutorial 1 and the classification-based agent from Assignment 1.

Approach	**Method**	**Performance**	**Std**[†]
Rule-based agent (Tut. 1)	Hand-crafted heuristic	≈ 85% success	—
ML agent – iter. 1 (Ass. 1)	Decision Tree (20k inst.)	≈ 15% success	—
ML agent – iter. 2 (Ass. 1)	Decision Tree (80k inst.)	≈ 50% success	—
Q-learning VQQL $K = 128$	Tabular RL (this work)	mean reward = +82.5	151.5

[†] Standard deviation of episode reward over 10 PLAY-mode episodes ($\varepsilon = 0$). Success-rate metrics for the rule-based and ML agents are taken from Tutorial 1 and Assignment 1 respectively and are not directly comparable to the mean-reward metric used for the Q-learning agent.

Table 6: Comparison of all approaches. Performance metrics differ across methods and should be interpreted accordingly (see footnote).

Q1. How does the RL agent compare to the classification-based agent in terms of performance? The best Q-learning model (VQQL $K = 128$) achieves a higher mean reward than both iterations of the behavioral cloning agent from Assignment 1. The second-iteration ML agent reaches ≈ 50% success rate, while the Q-learning agent — evaluated under the raw Gymnasium reward — obtains a mean of +82.5 with several episodes above +200, which corresponds to successful landings. The rule-based agent from Tutorial 1 remains the most consistent performer (≈ 85% success), but it relies on hand-crafted thresholds that do not generalize to different gravity settings or initial conditions without manual re-tuning.

Q2. What are the main differences between learning a policy (RL) and predicting actions (classification)? In **behavioral cloning** (Assignment 1), the model learns to imitate a fixed dataset of state-action pairs. The policy is only as good as the demonstrations, and it suffers from covariate shift: once the agent deviates from the training distribution, it encounters unseen states and fails cascadingly. The first-iteration ML agent illustrated this dramatically, achieving 92% offline accuracy but only 15% online success.

In **reinforcement learning**, the agent learns through direct interaction with the environment, optimizing cumulative reward rather than reproducing demonstrations. It can discover strategies not present in any dataset and is naturally exposed to the consequences of its own decisions, which eliminates covariate shift. The trade-off is that RL requires

many more interactions (5 000 episodes vs. a fixed dataset) and is sensitive to reward design and state representation.

Q3. Which approach generalizes better to unseen states, and why? The RL agent generalizes better to unseen states. Because it learns by exploring the environment rather than memorizing demonstrations, its Q-table encodes values for states encountered during its own trajectories, including states that a human demonstrator might never visit. The behavioral cloning agent, by contrast, assigns arbitrary predictions to states outside the training distribution. This was confirmed in Assignment 1: quadrupling the dataset size only raised the success rate from 15% to 50%, while the RL agent trained from scratch reached comparable or better performance without any demonstrations.

Q4. In which situations would you prefer a supervised learning approach over reinforcement learning? Supervised learning is preferable when: (i) a high-quality demonstration dataset is readily available and the environment is too costly or dangerous to explore (e.g., medical robotics, autonomous driving); (ii) the reward function is difficult to design and demonstrations implicitly encode the desired behavior; (iii) fast deployment is required, since behavioral cloning trains on a fixed dataset in a single pass whereas RL requires thousands of environment interactions; (iv) the state space is well covered by the demonstrations and covariate shift is not a concern.

3.3 Step 3: Discussion and Insights

Q1. What were the main challenges when applying Q-learning to a continuous environment? The fundamental challenge is that tabular Q-learning requires a finite, discrete state space, but the LunarLander provides a continuous 8-dimensional observation. Both discretization approaches introduced approximation errors: the manual bins merged states that require different actions, and the VQQL centroids could not adapt once training began. A second challenge was the high variance of episode returns due to random initial conditions, which made it difficult to assess convergence from individual rewards. A third challenge was reward design: the shaped reward added per-step penalties that accumulated over long episodes, inadvertently encouraging early termination.

Q2. How does state representation influence the success of reinforcement learning? State representation determines which distinctions the agent can make and which it cannot. A coarse representation (few bins or few clusters) forces the agent to take the same action in situations that should be treated differently, capping policy quality. An overly fine representation leaves most states unvisited, keeping their Q-values at zero and producing erratic behavior. The experiments in this assignment showed that the optimal representation for 5 000 training episodes is VQQL with $K = 128$: coarse enough to be well populated, fine enough to distinguish critical near-landing situations.

Q3. What are the main limitations of tabular Q-learning in this problem? Three limitations stand out. First, **scalability**: the Q-table grows with the number of states and actions; for continuous problems a finite table is always an approximation. Second, **sample efficiency**: with 8 100 manual states, many og them were never visited after 5 000 episodes, meaning a large portion of the table remains uninformed. Third, **fixed representation**: both the manual bins and the VQQL centroids are computed

before training and cannot adapt as the agent learns; states that are distinct from the agent's perspective may be merged, and no mechanism exists to refine the representation online.

Q4. How could this approach be improved? Several directions are possible. **Function approximation** (e.g., Deep Q-Network) would replace the table with a neural network that generalizes across similar states without requiring explicit discretization. **Online state refinement** could split or merge clusters during training based on the TD error, concentrating resolution where the value function changes rapidly. **Better exploration strategies** such as UCB or count-based bonuses would ensure more uniform coverage of the state space. Finally, **reward shaping** could be improved by using potential-based shaping, which is guaranteed not to change the optimal policy, unlike the additive penalties used in Phase 1.

Conclusions

Throughout this assignment we learned that, in tabular Q-learning, the choice of state representation matters far more than any other design decision. Our manual discretization built 8 100 states using domain knowledge, but with only 5 000 training episodes a large fraction of them never get visited, leaving much of the Q-table empty and the agent behaving erratically in unfamiliar situations. VQQL fixed this almost for free: by placing centroids where real trajectories actually go, just 128 states were enough to clearly outperform the manual approach. Going up to 256 was a bit surprising, since we expected more granularity to help, but the Q-table ended up underpopulated and PLAY-mode performance got worse. That non-monotonic behaviour was probably the most interesting lesson of the project, the bias-variance trade-off applied to state-space design.

Reward shaping was the other place where our intuition failed us. The penalties we added looked reasonable on paper, but they accumulated step after step and ended up pushing the agent towards finishing episodes quickly rather than landing well. We only noticed it after evaluating both agents under the same raw reward, which taught us not to trust a shaping function until we have actually measured it.

Comparing the RL agent with the behavioural-cloning model from Assignment 1 was also revealing. The supervised model had great offline accuracy but collapsed online due to covariate shift, while the RL agent trained on its own trajectories and avoided that problem by construction. The rule-based agent from Tutorial 1 still wins on consistency, but it only works because we tuned its thresholds for one specific setting, something the RL agent does automatically.

The clearest limitation we hit is that tabular Q-learning simply does not scale to a truly continuous state space, no matter how clever the discretization is. A natural next step would be replacing the table with a neural network (DQN), which would let the agent generalize between similar states without us having to design the representation beforehand. Even so, finishing the assignment with a simple table that actually lands the spacecraft was a satisfying way to see how representation, reward, and algorithm have to work together for any of this to work at all.